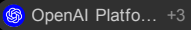


Worked for 13s

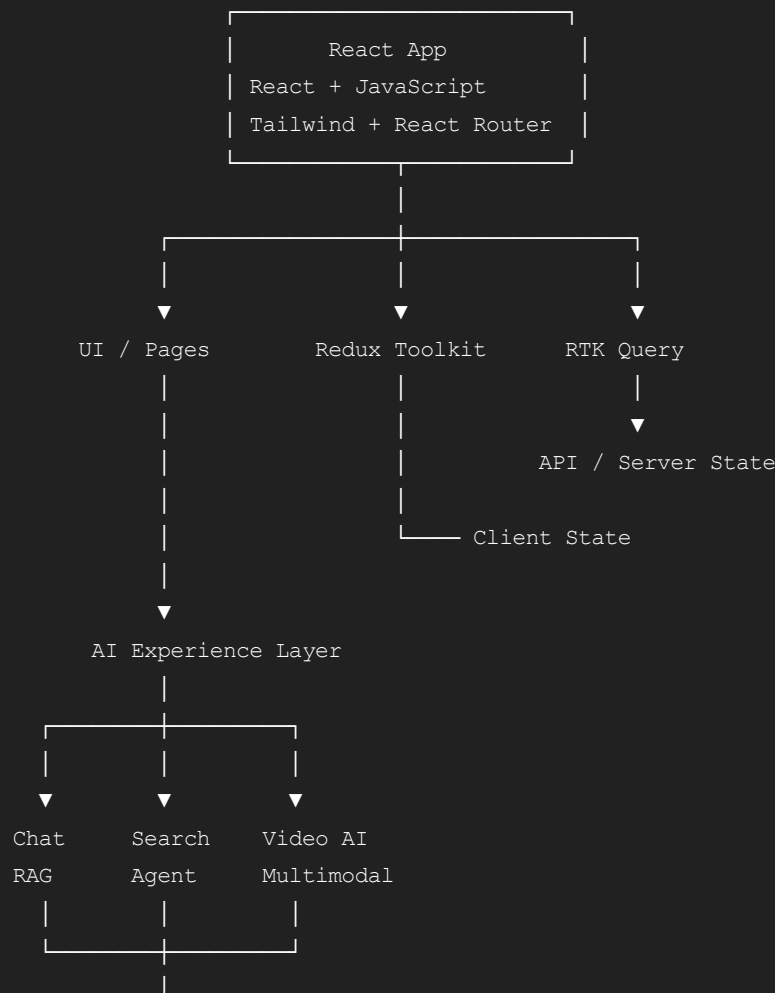
Yes—with one caveat: I can open the shared ChatGPT URL, but that shared page does not expose the underlying transcript to my browser environment, so I cannot honestly claim I read every message inside it. 

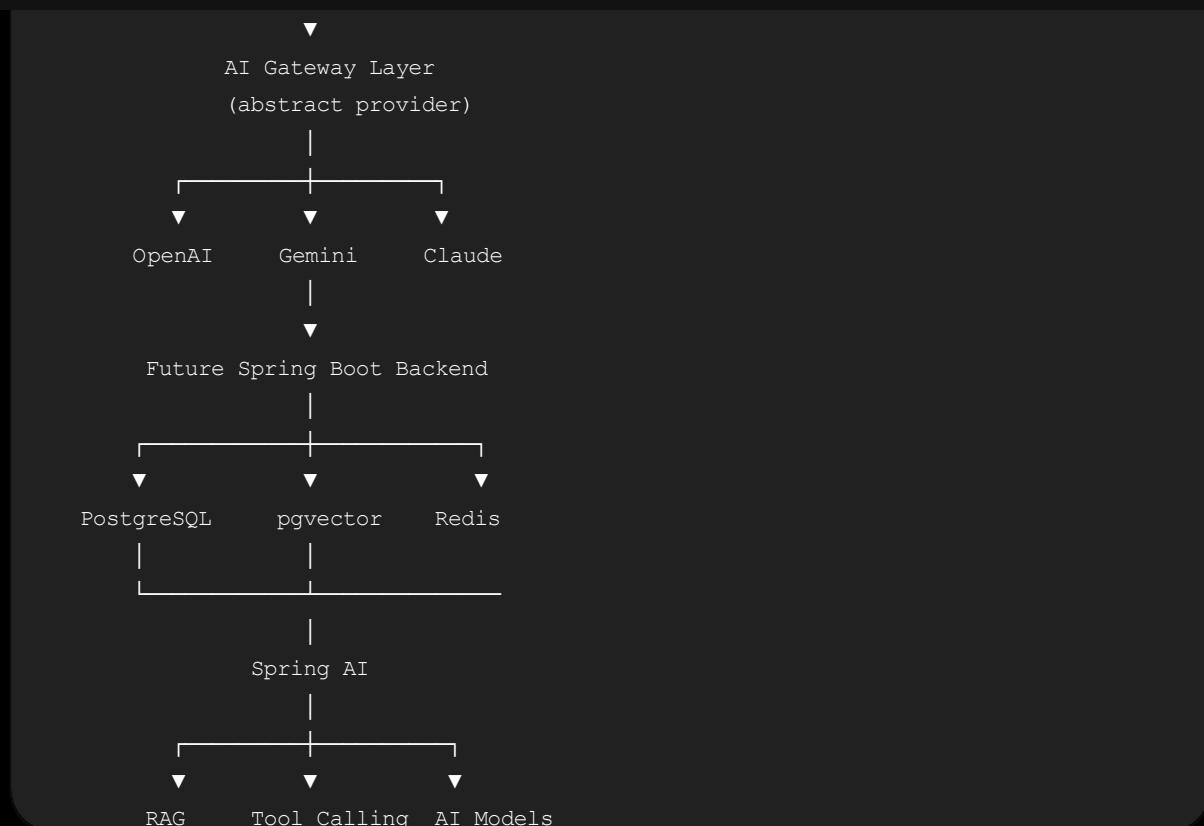
I **do** have the relevant context from our conversation here: you are building a YouTube clone with **React + JavaScript + Tailwind CSS + React Router + Redux Toolkit/Zustand**, and you want it to be strong enough for a **4-year Software Engineer preparing for a job switch**, with your existing **OTT/web/TV-platform experience** as an advantage.

Based on that, I would not build this as a normal "YouTube clone + a chatbot." I would build it as a **production-style video platform with an AI application layer**.

Current AI application patterns that are especially valuable here are **RAG, tool/function calling, structured outputs, streaming, multimodal processing, and AI observability/evaluation**. OpenAI's current APIs support tools, structured JSON-schema outputs and streaming; Spring AI provides corresponding abstractions for tool calling, RAG, vector stores, streaming, structured outputs, observability and MCP. 

1. The architecture I recommend





The critical architectural decision is this:

Do not let React components know which AI provider you are using.

Your components should ask your application for:

```
generateSummary()
askVideoQuestion()
searchVideosSemantically()
generatePlaylist()
generateQuiz()
moderateComment()
```

rather than:

```
callOpenAI()
```

That gives you a clean migration path from:

```
React → AI Provider
```

to:

```
React → Spring Boot → Spring AI → AI Provider
```

2. Production-grade frontend structure

I would use a **feature/domain-oriented architecture**, rather than putting every component into one giant `components` directory.

```
youtube-ai/
|
├─ public/
|   ├── favicon.ico
|   └─ manifest.json
|
├─ src/
|   ├── app/
|   │   ├── App.jsx
|   │   ├── router.jsx
|   │   ├── store.js
|   │   ├── providers.jsx
|   │   └─ errorHandler.js
|   ├── assets/
|   │   ├── images/
|   │   ├── icons/
|   │   └─ fonts/
|   ├── components/
|   │   ├── ui/
|   │   │   ├── Button.jsx
|   │   │   ├── Input.jsx
|   │   │   ├── Dialog.jsx
|   │   │   ├── Dropdown.jsx
|   │   │   ├── Skeleton.jsx
|   │   │   ├── Spinner.jsx
|   │   │   └─ Tooltip.jsx
|   │   ├── layout/
|   │   │   ├── AppShell.jsx
|   │   │   ├── Header.jsx
|   │   │   ├── Sidebar.jsx
|   │   │   ├── MobileNavigation.jsx
|   │   │   └─ PageContainer.jsx
|   │   └─ video/
|   │       ├── VideoCard.jsx
|   │       ├── VideoGrid.jsx
|   │       ├── VideoPlayer.jsx
|   │       ├── VideoMetadata.jsx
|   │       ├── VideoActions.jsx
|   │       └─ VideoSkeleton.jsx
|   └─ features/
|       ├── auth/
|       │   ├── authSlice.js
|       │   └─ authSelectors.js
```

```

├── authApi.js
├── authHooks.js
├── components/
├── player/
│   ├── playerSlice.js
│   ├── playerSelectors.js
│   ├── playerHooks.js
│   └── components/
├── search/
│   ├── searchSlice.js
│   ├── searchSelectors.js
│   └── components/
├── library/
│   ├── librarySlice.js
│   ├── librarySelectors.js
│   └── components/
├── preferences/
│   ├── preferencesSlice.js
│   └── preferencesSelectors.js
├── ai/
│   ├── aiSlice.js
│   ├── aiSelectors.js
│   ├── aiApi.js
│   ├── aiTypes.js
│   ├── hooks/
│   │   ├── useVideoSummary.js
│   │   ├── useVideoQA.js
│   │   ├── useSemanticSearch.js
│   │   ├── usePlaylistGenerator.js
│   │   └── useVideoQuiz.js
│   ├── components/
│   │   ├── AIButton.jsx
│   │   ├── AIAssistant.jsx
│   │   ├── AIChat.jsx
│   │   ├── AISummary.jsx
│   │   ├── AIChapters.jsx
│   │   ├── AIAnswer.jsx
│   │   ├── AIQuiz.jsx
│   │   ├── AIRecommendation.jsx
│   │   └── AIThinkingIndicator.jsx
│   ├── services/
│   │   ├── aiGateway.js
│   │   ├── videoSummaryService.js
│   │   ├── semanticSearchService.js
│   │   ├── recommendationService.js
│   │   └── moderationService.js

```



```
|   └─ hooks/
|       └─ useDebounce.js
|       └─ useInfiniteScroll.js
|       └─ useLocalStorage.js
|       └─ useMediaQuery.js
|
|   └─ routes/
|       └─ ProtectedRoute.jsx
|       └─ PublicRoute.jsx
|       └─ RouteErrorBoundary.jsx
|       └─ routeConfig.js
|
|   └─ utils/
|       └─ constants.js
|       └─ formatNumber.js
|       └─ formatDuration.js
|       └─ formatDate.js
|       └─ validation.js
|
|   └─ lib/
|       └─ logger.js
|       └─ analytics.js
|
|   └─ App.jsx
|   └─ main.jsx
|   └─ index.css
|
└─ .env.example
└─ .gitignore
└─ eslint.config.js
└─ package.json
```

Why this structure?

The important distinction is:

```
pages
  ↓
compose application screens

components
  ↓
generic/reusable UI

features
  ↓
business functionality

api
  ↓
server-state/data fetching

services
  ↓
integration boundaries
```

```
hooks
  ↓
reusable React behavior

utils
  ↓
pure helper functions
```

And most importantly:

```
features/ai/
```

becomes a **first-class domain**, rather than scattering AI code throughout the application.

3. Redux Toolkit architecture

I would **not** put every API response into Redux manually.

Use Redux Toolkit for **client/application state** and RTK Query for **server state and caching**. RTK Query is specifically designed for data fetching and caching, while

`createSlice` handles domain state and actions. [redux-toolkit.js... +1](#)

Store

```
Redux Store
|
├─ auth
├─ ui
├─ player
├─ search
├─ library
├─ preferences
├─ ai
├─
└─ api
    ├─ videos
    ├─ channels
    ├─ comments
    └─ AI endpoints
```

4. `authSlice`

Responsibility

Authentication and current-user client state.

JavaScript

```
{  
  user: null,  
  isAuthenticated: false,  
  status: "idle",  
  error: null  
}
```

Actions

```
loginStarted  
loginSucceeded  
loginFailed  
logout  
updateUser
```

Selectors

```
selectCurrentUser  
selectIsAuthenticated  
selectAuthStatus
```

Do not store sensitive tokens in arbitrary Redux state if you eventually have a backend. Prefer an HTTP-only secure cookie architecture once Spring Boot exists.

5. `uiSlice`

Responsible only for global UI state.

JavaScript

```
{  
  sidebar: {  
    isOpen: true  
  },  
  
  modal: {  
    active: null  
  },  
  
  theme: "dark"  
}
```

Actions:

```
toggleSidebar  
openModal
```



```
closeModal  
setTheme
```

Selectors:

```
selectSidebarOpen  
selectActiveModal  
selectTheme
```

6. `playerSlice`

This is especially relevant to an OTT engineer.

JavaScript

```
{  
  currentVideoId: null,  
  isPlaying: false,  
  volume: 1,  
  isMuted: false,  
  playbackRate: 1,  
  currentTime: 0,  
  duration: 0  
}
```

Actions:

```
setCurrentVideo  
play  
pause  
setVolume  
toggleMute  
setPlaybackRate  
setCurrentTime  
setDuration  
resetPlayer
```

This gives you a nice opportunity to demonstrate that you understand playback state independently from video metadata.

7. `searchSlice`

I would keep this deliberately small.

JavaScript

```
{  
  query: "",
```

```
mode: "keyword",
filters: {
  duration: null,
  uploadDate: null,
  category: null
}
```

But there is an important architectural decision:

Do not store search results here.

Store them in RTK Query.

Also, use the URL as the source of truth for shareable search state:

```
/search?q=react&duration=medium
```

Redux handles transient search UI/preferences; React Router handles navigational state.

8. `librarySlice`

For local/user interaction state:

JavaScript

```
{
  likedVideoIds: [],
  savedVideoIds: [],
  historyEnabled: true
}
```

Later, these become backend-backed resources:

```
POST /api/v1/users/me/videos/{id}/like
POST /api/v1/users/me/watch-later
GET /api/v1/users/me/history
```

At that point RTK Query should become the main source of truth.

9. `preferencesSlice`

JavaScript

```
{
  autoplay: true,
  language: "en",
  captionsEnabled: true,
  preferredCategories: [],
}
```

```
ai: {
  enabled: true,
  responseStyle: "concise"
}
```

This is useful because your AI features can respect the user's preferences.

10. The important slice — `aiSlice`

This is where I would be careful.

Do **not** dump all AI-generated content into Redux.

Instead, Redux manages **AI session/UI state**, while RTK Query or a dedicated AI service manages generated/server state.

JavaScript

```
{
  assistant: {
    isOpen: false,
    activeConversationId: null,
    mode: "video"
  },

  request: {
    status: "idle",
    feature: null,
    requestId: null,
    error: null
  },

  context: {
    videoId: null,
    timestamp: null
  },

  streaming: {
    isStreaming: false
  }
}
```

Actions:

```
openAssistant
closeAssistant
setAssistantMode
setVideoContext
startAIRequest
completeAIRequest
failAIRequest
```

```
startStreaming
stopStreaming
```

Selectors:

```
selectAssistantOpen
selectAIStatus
selectAIContext
selectAIStreaming
selectActiveAIFeature
```

Redux Toolkit now also supports defining selectors directly inside `createSlice`, which can make slice APIs cleaner in modern Redux architectures. [redux-toolkit.js... +1](#)

11. RTK Query architecture

I'd create a single base API:

```
src/api/baseApi.js
```

with separate endpoint modules:

```
api/
├─ baseApi.js
└─ endpoints/
   ├─ videosApi.js
   ├─ channelsApi.js
   ├─ commentsApi.js
   ├─ libraryApi.js
   └─ aiApi.js
```

Conceptually:

```
baseApi
├─
├─ videos
├─ channels
├─ comments
├─ users
└─ ai
```

RTK Query gives generated hooks and cache selectors for endpoints, which is preferable to creating dozens of manual `loading/error/data` slices. [redux-toolkit.js... +1](#)

12. React Router architecture

Use `createBrowserRouter`.

```

/
|
├─ /watch/:videoId
├─ /search
├─ /channel/:channelId
|
├─ /library
|   └─ /history
|   └─ /liked
|   └─ /watch-later
|
├─ /ai
|   └─ /assistant
|   └─ /recommendations
|   └─ /studio
|
├─ /login
|
└─ *

```

The route hierarchy:

```

AppRoot
|
└─ AppShell
    |
    ├─ Header
    ├─ Sidebar
    |
    └─ Outlet
        |
        ├─ Public
        |   └─ Home
        |   └─ Watch
        |   └─ Search
        |   └─ Channel
        |
        ├─ Protected
        |   └─ Library
        |   └─ History
        |   └─ Watch Later
        |   └─ AI Studio
        |
        └─ Error Boundary

```

Lazy loading

For a production-style portfolio application, lazy-load routes such as:

```

AI Studio
AI Assistant
Channel

```

React `Suspense` provides the loading boundary, while React Router's `route.lazy` allows heavier route implementations to be loaded only when navigated to.  React +1

Example:

JavaScript

```
{
  path: "ai/studio",
  lazy: () => import("../pages/AI/AIStudio.route"),
}
```

That is a better demonstration of application architecture than simply doing:

JavaScript

```
const AIStudio = lazy(() => import("../AIStudio"));
```

everywhere.

13. Route protection

PublicRoute

```
|
├─ /
├─ /watch/:videoId
├─ /search
└─ /channel/:channelId
```

ProtectedRoute

```
|
├─ /library
├─ /history
├─ /watch-later
└─ /ai/studio
```

The protected route should be responsible for **authorization**, not the individual page.

14. Route error architecture

Every major route branch should have an error boundary.

App

```
└─ Router
    └─ AppShell
        └─ Home
```

```
└─ Watch
└─ Search
└─ ErrorBoundary
```

You want graceful states for:

```
404
API failure
AI failure
rate limit
authentication expiry
video unavailable
network offline
```

For an AI application especially, an AI timeout should not take down the video player.

15. AI integration architecture

This is probably the most important part of the whole project.

I recommend:

```
React Component
  |
  ▼
AI Hook
  |
  ▼
AI Service
  |
  ▼
AI Gateway
  |
  ├── Browser/mock adapter
  |
  ├── Serverless adapter
  |
  └── Spring Boot adapter
```

For example:

JavaScript

```
useVideoSummary(videoId)
```

calls:

```
videoSummaryService
  ↓
aiGateway.generateSummary()
```

The component never knows whether the implementation is:

```
OpenAI
Gemini
Claude
Spring AI
Mock JSON
```

16. AI Gateway

Create:

```
features/ai/services/aiGateway.js
```

Conceptually:

JavaScript

```
const aiGateway = {
  summarizeVideo,
  askAboutVideo,
  semanticSearch,
  generatePlaylist,
  generateQuiz,
  generateCreatorMetadata,
  moderateComment,
  recommendVideos,
};
```

Then you can have:

```
aiGateway
├── mockProvider
├── openAIProvider
└── backendProvider
```

Today

```
React
↓
AI Gateway
↓
AI Provider
```

Later


```
React
↓
AI Gateway
↓
Spring Boot REST API
↓
Spring AI
↓
Model
```

That separation will save you a huge refactor later.

17. Security architecture

There is one thing I would **not** do in your production-style project:

JavaScript

```
const OPENAI_API_KEY = import.meta.env.VITE_OPENAI_API_KEY;
```

and call the model directly from browser code.

A `VITE_*` variable is part of the client bundle and should be treated as public.

For the portfolio prototype, you can use:

```
React
↓
Vercel/Netlify/Cloudflare serverless function
↓
AI provider
```

Then later replace the serverless function with:

```
Spring Boot
↓
Spring AI
↓
AI provider
```

This also gives you an opportunity to talk about **API-key isolation**, **rate limiting**, **authentication** and **cost control** during interviews.

18. Eight AI features I recommend

I would deliberately choose features that demonstrate **different AI engineering patterns**, rather than eight variations of "ask ChatGPT."

#	Feature	Main AI concept	Difficulty
1	Video Intelligence Copilot	RAG + structured output	High
2	Semantic Video Search	Embeddings + retrieval	High
3	AI Learning Mode	RAG + generation	Medium/High
4	Multimodal Video Q&A	Multimodal AI	High
5	AI Playlist & Recommendation Agent	Tool calling + personalization	Very High
6	Creator Copilot	Structured generation	Medium
7	AI Comment Intelligence	Classification + moderation	Medium
8	AI Dubbing / Accessibility	Speech AI + translation + TTS	High

These map particularly well to capabilities supported by current AI platforms and Spring AI: embeddings/vector stores, RAG, structured outputs, audio/transcription, tool calling and multimodal models.  Home +1

19. Feature #1 — AI Video Intelligence Copilot

What it does

On every video page:

✨ AI Assistant

[Summarize]

[Key Takeaways]

[Explain This Video]

[Ask a Question]

It can generate:

Executive Summary

Key Concepts

Important Moments

Technical Terms

Chapter Breakdown
Beginner Explanation
Advanced Explanation

Then the user can ask:

"What did the speaker say about Redux?"

The AI answers using the video's transcript/context.

Why recruiters care

This demonstrates **RAG**, contextual generation, structured outputs, streaming, and conversational UX.

It is much more meaningful than putting a generic chatbot in the sidebar.

RAG is specifically designed to provide models with relevant external context instead of relying only on model memory. Spring AI provides first-class RAG/vector-store abstractions for this pattern.  Home +1

UX

```
Video
|
├─ Transcript
├─ AI Summary
├─ Chapters
└─ Ask AI
```

Frontend architecture

```
Watch.jsx
|
└─ VideoAIPanel
    ├── AISummary
    ├── AIChapters
    └─ AIChat
```

Redux

```
ai.context.videoId
ai.assistant.mode
ai.request.status
ai.streaming.isStreaming
```

Hooks

```
useVideoSummary()
useVideoQA()
```

```
useVideoChapters()
```

No-backend version

Use a local/mock transcript:

```
videoId → transcript JSON → AI gateway
```

You could also demonstrate the feature against a small set of videos with preloaded transcript data.

Future Spring Boot

```
React
↓
POST /api/v1/ai/videos/{videoId}/summary
↓
Spring Boot
↓
Transcript service
↓
Embedding / Vector Store
↓
Spring AI RAG
↓
LLM
```

Technologies

```
OpenAI / Gemini / Claude
Embeddings
pgvector
Spring AI
SSE
Structured Outputs
```

OpenAI supports streaming responses and structured JSON-schema responses, both useful for rendering progressive AI UI reliably. [OpenAI Platfo...](#)

Difficulty

8/10

Resume

Built an AI-powered video intelligence assistant using RAG and structured LLM outputs to generate summaries, chapters, key takeaways and contextual Q&A from video transcripts.

20. Feature #2 — Semantic Video Search

This is one of my strongest recommendations.

Instead of:

```
"react hooks"
```

the user can search:

```
"Find beginner-friendly videos explaining state management with practical examples."
```

The system understands the **meaning**, not simply matching keywords.

Why recruiters care

This demonstrates:

```
Embeddings
Vector search
Semantic retrieval
Ranking
Query transformation
Search UX
```

Vector databases are specifically designed for similarity-based retrieval and are commonly used as the retrieval layer of RAG systems. [👉 Home](#)

UX

🔍 Ask naturally

```
"Videos that explain Redux Toolkit
with a real-world project"
```

↓

```
AI understands intent
```

↓

```
Semantic results
```

Frontend

```
SemanticSearchBar
  ↓
useSemanticSearch()
  ↓
semanticSearchService
```

Redux

Very little.

```
search.query
search.mode = "semantic"
search.filters
```

Search results stay in RTK Query/server state.

No backend

Create a local catalogue of perhaps 100 videos with:

```
title
description
category
transcript
tags
```

Generate embeddings once and store them locally for the demo, or use a hosted vector service behind a safe server-side proxy.

Future backend

```
React
↓
Spring Boot
↓
Embedding Service
↓
pgvector / Qdrant / Pinecone
↓
Top-K videos
↓
Ranking
↓
React
```

Spring AI supports multiple vector stores and metadata filtering, so the underlying provider can change without forcing a frontend redesign.  Home +1

Difficulty

8/10

Resume

Implemented semantic video search using embedding-based retrieval and vector similarity ranking, enabling natural-language discovery beyond keyword matching.

21. Feature #3 — AI Learning Mode

This one would be particularly strong for your portfolio.

When watching an educational video:

 Learning Mode

The AI creates:

```
Summary
↓
Key Concepts
↓
Flashcards
↓
Quiz
↓
Practice Questions
↓
Knowledge Check
```

Example:

"Generate a 5-question quiz based only on this video."

The user answers.

The AI evaluates their answers.

Why recruiters care

This demonstrates:

```
RAG
Structured outputs
State machines
Adaptive UI
Evaluation
Personalization
```

Frontend

```
LearningMode
├─ Summary
├─ Flashcards
├─ Quiz
├─ Progress
└─ Results
```

Redux

JavaScript

```
{
  learning: {
    videoId: null,
    mode: "quiz",
    currentQuestion: 0,
    answers: {},
    score: null
  }
}
```

AI output should be structured

For example:

JSON

```
{
  "questions": [
    {
      "question": "...",
      "options": ["...", "...", "...", "..."],
      "correctAnswer": 2,
      "explanation": "..."
    }
  ]
}
```

Structured outputs are preferable when your UI needs predictable machine-readable data instead of parsing free-form model text.  OpenAI Platfo...

Future

Spring Boot can store:

```
learning_sessions
quiz_attempts
scores
weak_topics
```

Then the AI can adapt future quizzes.

Difficulty

7/10

Resume

Built an AI learning mode that transforms video transcripts into structured summaries,

flashcards and adaptive quizzes with topic-based progress tracking.

22. Feature #4 — Multimodal Video Q&A

This is where your project starts becoming genuinely interesting.

Instead of only understanding the transcript, AI can answer questions about **what appears in the video**.

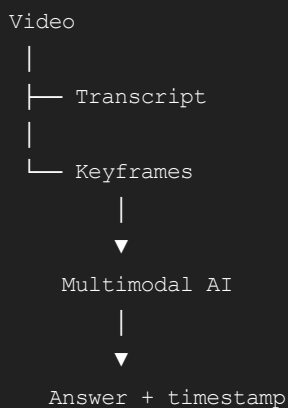
For example:

"What architecture diagram does the presenter show?"

or:

"At what point does the presenter demonstrate Redux middleware?"

Pipeline



UX

AI Answer

"The Redux middleware diagram appears around 08:42."

[▶ Jump to 08:42]

That `Jump to timestamp` interaction is excellent because it combines AI with actual application functionality.

Redux

```
ai.context.videoId  
ai.context.timestamp
```

No backend

Use a curated set of:

```
video
transcript
keyframe images
timestamps
```

Future backend

A media-processing pipeline can generate keyframes:

```
Video
↓
FFmpeg / Media Processing
↓
Keyframe Extraction
↓
Embeddings / Vision Model
↓
Vector Store
↓
Spring AI
```

Difficulty

9/10

Resume

Developed a multimodal video Q&A experience combining transcript context and video keyframes to answer visual questions and navigate users directly to relevant timestamps.

23. Feature #5 — AI Playlist & Recommendation Agent

This is where I would introduce **tool calling**.

Instead of merely asking:

"Recommend videos."

let the user say:

"Create a 2-week React learning playlist for me, starting from fundamentals and ending with advanced state management."

The AI can call application tools such as:

```
searchVideos()  
getVideoDetails()  
getUserHistory()  
getLikedVideos()  
createPlaylist()  
addVideoToPlaylist()
```

Tool calling is a particularly useful modern AI application pattern because the model can request application-defined operations, while the application remains responsible for executing them. Spring AI explicitly supports this model.  Home +1

Architecture

```
User  
  ↓  
AI Assistant  
  ↓  
LLM  
  ↓  
Tool Call  
  ├── searchVideos()  
  ├── getHistory()  
  └── createPlaylist()  
      ↓  
Application  
      ↓  
Result  
      ↓  
LLM  
      ↓  
Final Plan
```

Frontend

```
AIAssistant  
AICommandBar  
AIActionPreview  
AIPlaylistCard
```

Redux

```
ai.activeTask  
ai.pendingToolAction  
ai.requiresConfirmation  
ai.taskStatus
```

Important security behavior

The AI should **not** directly mutate important user resources.

Use:

```
AI → proposes action
    ↓
User confirmation
    ↓
Application executes action
```

For example:

```
AI wants to create:

"React Mastery - 12 videos"

[Cancel] [Create Playlist]
```

That is a much more mature agentic UX.

Future backend

Spring AI can own tool definitions such as:

```
Java

@Tool
searchVideos(...)

@Tool
getWatchHistory(...)

@Tool
createPlaylist(...)
```

Spring AI's current tool-calling architecture is explicitly designed around this model.



Difficulty

9/10

Resume

Designed an agentic video discovery assistant using LLM tool calling to search content, analyze viewing history and generate user-approved playlists.

24. Feature #6 — Creator Copilot

Give creators an AI assistant:

Creator Studio

Input:

Video transcript

Output:

Title options
Description
Tags
Chapters
Short-form hooks
Thumbnail concepts
Audience
SEO keywords

Example

Transcript:
"Today we will learn Redux Toolkit..."

AI generates:

Title:
"Redux Toolkit Explained With a Real React Project"

Chapters:
00:00 Introduction
02:14 Redux basics
05:32 createSlice
...

Why recruiters care

This demonstrates **structured generation**, not just chat.

Frontend

```
AIStudio
├─ TranscriptInput
├─ GenerationControls
├─ TitleSuggestions
├─ DescriptionEditor
├─ ChapterEditor
└─ SEOInsights
```

Backend migration

POST /api/v1/ai/creator/generate

Spring AI can map structured outputs to application data/POJOs, which fits this kind of workflow well. [Home](#)

Difficulty

6/10

Resume

Built an AI Creator Studio that generates structured video metadata, chapters, SEO descriptions and title variations from transcript content.

25. Feature #7 — AI Comment Intelligence

Don't just build:

"AI checks bad words."

Make it a proper moderation/intelligence system.

For every comment:

Toxicity
Sentiment
Intent
Spam likelihood
Harassment
Question detection

Then:

 AI Insights

Positive: 72%
Questions: 18%
Spam: 4%
Toxic: 1%

Creators can filter:

[Questions]
[Negative feedback]
[Feature requests]
[Spam]

Why recruiters care

This demonstrates **classification**, structured AI output, moderation and analytics.

Frontend

```
CommentSection
├─ Comment
├─ AICommentBadge
├─ CommentFilters
└─ CommentInsights
```

Future backend

```
POST /api/v1/ai/comments/moderate
```

At scale, the moderation model belongs on the server so that users cannot circumvent moderation.

Difficulty

6/10

Resume

Implemented AI-powered comment intelligence for toxicity, spam, sentiment and intent classification, enabling creators to surface actionable audience feedback.

26. Feature #8 — AI Dubbing & Accessibility

This is the feature I'd choose specifically because of your OTT background.

Build:

```
🌐 AI Accessibility
```

Features:

```
Automatic captions
Translation
Translated subtitles
Text-to-speech
Audio language selection
```

Eventually:

```
English
Hindi
Marathi
Spanish
```

French
Japanese

Why this is especially strong for you

This isn't an arbitrary AI feature.

It connects your existing **OTT/video-platform experience** to modern AI.

You can discuss:

```
video playback
captions
media timelines
accessibility
localization
audio tracks
AI transcription
translation
TTS
```

That creates a much stronger interview story than "I made a chatbot."

Architecture

```
Video
↓
Speech-to-Text
↓
Transcript
↓
Translation
↓
Subtitle Track
↓
Video Player
```

For example:

```
English Audio
↓
Speech Recognition
↓
Transcript
↓
Translation
↓
WebVTT
↓
<track>
```

Future backend


```
Video
↓
Media Processing Pipeline
↓
Speech-to-Text
↓
Translation
↓
Subtitle Storage
↓
CDN
↓
Player
```

Spring AI supports audio transcription and text-to-speech model abstractions through its model API layer.  Home +1

Difficulty

8/10

Resume

Built an AI-powered accessibility pipeline for automated transcription, multilingual subtitle generation and localized video playback.

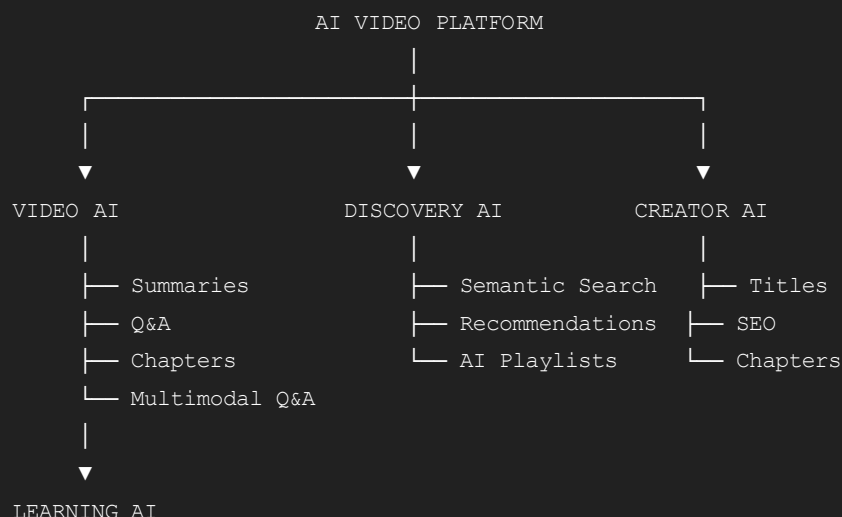
27. How the eight features fit together

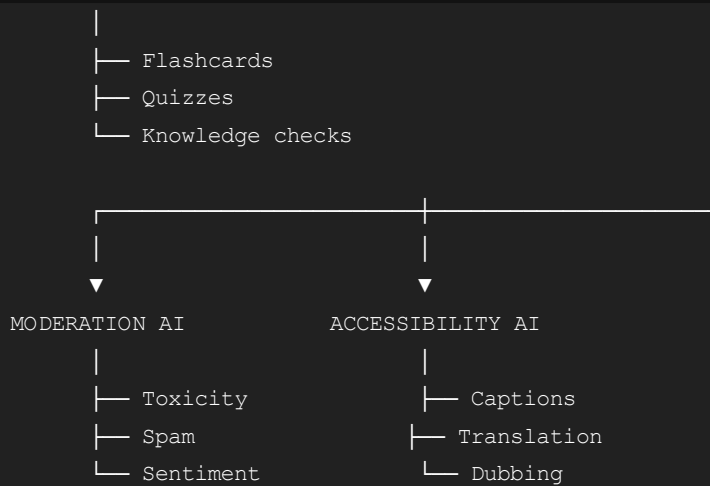
This is the part that can make your portfolio stand out.

Don't present them as:

8 unrelated AI buttons

Present them as one cohesive **AI Video Intelligence Platform**.





That's a **platform**, not simply a clone.

28. AI feature → architecture mapping

Feature	Redux	Hooks	Components	AI service	Backend
Video Copilot	AI session	<code>useVideoQA</code>	AIChat	RAG service	Spring /
Semantic Search	Search	<code>useSemanticSearch</code> <code>h</code>	Search UI	Embedding service	Vector I
Learning Mode	Learning state	<code>useVideoQuiz</code>	Quiz/Flashcards	Quiz service	RAG
Multimodal Q&A	AI context	<code>useVideoVision</code>	Vision panel	Vision service	Media p
AI Playlist Agent	AI task state	<code>useAIAgent</code>	Action preview	Agent gateway	Tool call
Creator Copilot	Studio state	<code>useCreatorAI</code>	AI Studio	Generation service	Spring /
Comment Intelligence	Filters	<code>useCommentAI</code>	AI badges	Moderation service	Moderation
AI Accessibility	Player state	<code>useCaptionsAI</code>	Caption UI	Audio/translation service	Media p

29. The future Spring Boot architecture

When you are ready to build the backend, I would use:

```

youtube-backend/
|
├─ src/main/java/com/yourname/youtube/
|
├─ auth/
├─ user/
├─ video/
├─ channel/
├─ comment/
├─ playlist/
├─ history/
|
├─ ai/
|   ├─ controller/
|   ├─ service/
|   ├─ model/
|   ├─ prompt/
|   ├─ tool/
|   ├─ rag/
|   ├─ embedding/
|   ├─ moderation/
|   └─ recommendation/
|
├─ media/
|   ├─ transcription/
|   ├─ subtitles/
|   ├─ translation/
|   └─ keyframes/
|
├─ vector/
|
└─ infrastructure/
    ├─ security/
    ├─ caching/
    ├─ storage/
    └─ observability/

```

Spring AI currently provides abstractions for model access, vector stores, RAG, tool calling, structured outputs, memory, observability and MCP, making it a particularly natural fit for the backend you described.  Home +2

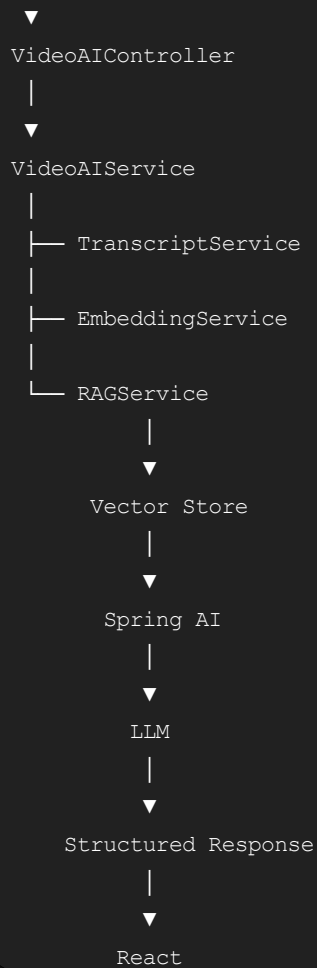
30. Future AI backend flow

For the Video Copilot:

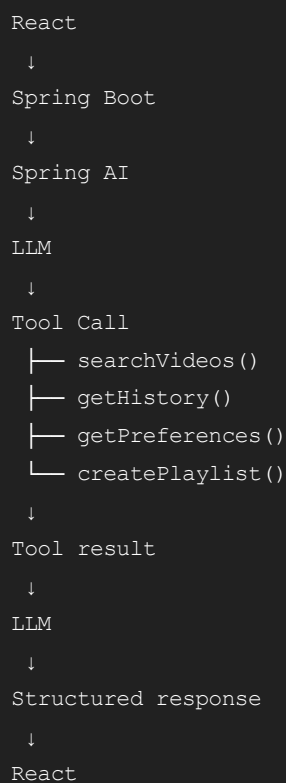
```

React
|
| POST /api/v1/ai/videos/123/ask
▼
Spring Boot
|

```



For agentic actions:



This is precisely the type of application-owned tool execution that Spring AI's tool-calling architecture supports. [🏠 Home](#)

31. What I would build first

Do **not** attempt all eight simultaneously.

I'd build them in this order:

```
PHASE 1 - Foundation
|
├─ React architecture
├─ React Router
├─ Redux Toolkit
├─ RTK Query
├─ YouTube data
└─ Video player
```

Then:

```
PHASE 2 - First AI layer
|
├─ AI Gateway
├─ Video Summary
├─ AI Q&A
└─ Structured outputs
```

Then:

```
PHASE 3 - Retrieval
|
├─ Transcript ingestion
├─ Embeddings
├─ Semantic Search
└─ RAG
```

Then:

```
PHASE 4 - Intelligence
|
├─ Learning Mode
├─ AI Recommendations
├─ AI Playlist Agent
└─ Creator Copilot
```

Then:

```
PHASE 5 - Advanced
|
├─ Multimodal Q&A
├─ Comment Intelligence
└─ AI Accessibility
```

This gives you a natural progression from:

React developer → frontend engineer → AI-enabled application engineer.

32. What I would highlight in your GitHub README

Your project shouldn't be called simply:

YouTube Clone

I'd position it as something closer to:

Vidora — AI-Powered Video Intelligence Platform

Then explain:

Vidora is a production-style YouTube-inspired video platform built with React, Redux Toolkit, React Router and Tailwind CSS, enhanced with RAG, semantic search, multimodal video intelligence, agentic recommendations, AI learning tools and accessibility features.

The frontend uses a provider-agnostic AI gateway designed to transition from a browser/serverless prototype to a Java Spring Boot + Spring AI backend.

That sentence communicates much more engineering depth than:

"I built a YouTube clone using React."

33. Your resume positioning

For a 4-year engineer, I would **not** describe this like a beginner project.

A strong project section could eventually look like:

Vidora — AI-Powered Video Intelligence Platform

React, JavaScript, Redux Toolkit, RTK Query, React Router, Tailwind CSS, Spring Boot, Spring AI, Vector Search

Then bullets such as:

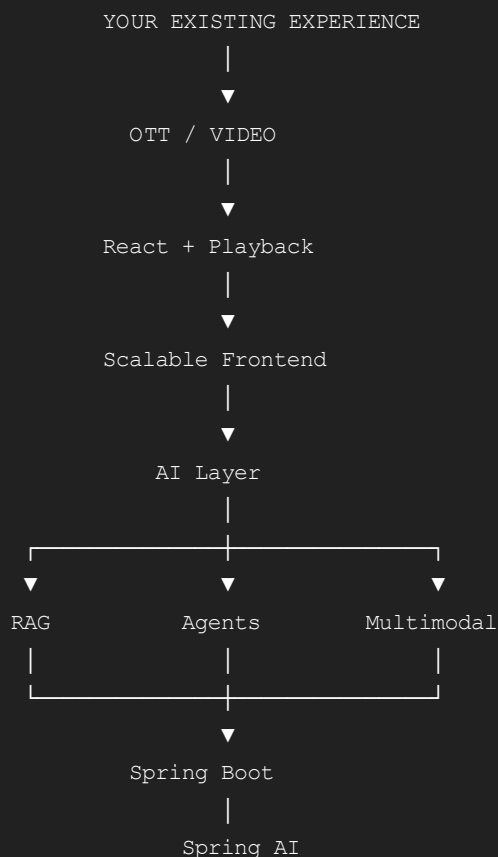
- Architected a production-style video platform using React, Redux Toolkit, RTK Query and nested React Router layouts, separating client state, server state, URL state and AI application state.
- Built an AI video intelligence layer using RAG and structured LLM outputs to generate contextual summaries, chapters, key takeaways and transcript-grounded Q&A.

- Implemented semantic video discovery using embeddings and vector similarity search, enabling natural-language content retrieval beyond keyword matching.
- Developed an agentic playlist assistant using tool calling to search videos, analyze user context and generate user-approved personalized playlists.
- Built AI learning workflows that transform video content into structured quizzes, flashcards and knowledge checks.
- Designed a provider-agnostic AI gateway enabling the frontend AI layer to migrate from a prototype implementation to a Java/Spring Boot + Spring AI backend.

34. The biggest recommendation I have for your particular profile

Because you're already working in **OTT and TV applications**, don't try to compete with other candidates by simply making the most flashy AI demo.

Your strongest portfolio narrative is:



That gives you a very coherent interview story:

"I already understand video/OTT applications. I used this project to demonstrate how I would architect a modern AI-enabled video platform end-to-end."

That is substantially stronger than:

"I made a YouTube clone and added eight AI buttons."

My recommended final stack

Frontend

React
JavaScript
Tailwind CSS
React Router
Redux Toolkit
RTK Query
Vite

AI Application Layer

AI Gateway
Structured Outputs
Streaming / SSE
RAG
Embeddings
Vector Search
Tool Calling
Multimodal AI

Prototype AI Infrastructure

Serverless API / AI Provider
OpenAI / Gemini / Claude
Vector Store

Future Backend

Java
Spring Boot
Spring Security
Spring AI
PostgreSQL
pgvector
Redis
Object Storage
FFmpeg / Media Processing

One particularly good strategic decision is to make **RAG + semantic search + tool calling + multimodal video intelligence** the four architectural pillars. Those patterns transfer well

beyond a YouTube clone and map cleanly to the capabilities exposed by current AI frameworks, including Spring AI.  Home +2

For your resume, I would prioritize **quality and technical depth over actually shipping all eight**. A polished implementation of the first 4–5 features, with good architecture, error handling, streaming, caching, tests and a strong README, will impress more than eight shallow integrations.